

AuthiChain User Onboarding System Guide

Overview

The AuthiChain User Onboarding System is a comprehensive solution designed to guide new users through their first experiences on the platform. It combines automated email sequences, interactive tutorials, progress tracking, and contextual help to ensure users quickly understand and engage with key features.

Table of Contents

1. [System Components](#)
 2. [Email Templates](#)
 3. [In-App Tutorial](#)
 4. [Progress Tracking](#)
 5. [Getting Started Page](#)
 6. [Implementation Guide](#)
 7. [Customization](#)
 8. [Analytics & Monitoring](#)
-

System Components

The onboarding system consists of five main components:

1. Automated Email Sequences

- Welcome email upon registration
- Subscription confirmation email
- First NFT minting success email
- Follow-up reminder emails (future enhancement)

2. Interactive Tutorial (React Joyride)

- Step-by-step platform walkthrough
- Contextual tooltips and guides
- Can be skipped or restarted
- Progress automatically saved

3. Onboarding Checklist

- Visual progress indicator
- Five key onboarding tasks
- Contextual action buttons
- Celebration screen on completion

4. Progress Tracking System

- Database-backed status tracking

- Real-time updates via API
- User preference storage
- Analytics-ready data

5. Getting Started Page

- Comprehensive user guides
 - Video tutorial placeholders
 - Step-by-step instructions
 - FAQ sections
 - Quick action links
-

Email Templates

Location

All email templates are located in `/app/emails/`

Available Templates

1. Welcome Email (`welcome-email.tsx`)

Purpose: Sent immediately after user registration

Variables:

- `username` (string): User's display name
- `walletAddress` (string, optional): User's connected wallet address

Content Includes:

- Welcome message
- Getting started steps
- Platform benefits overview
- Resource links
- Support contact information

Sample Content:

Subject: Welcome to AuthiChain - Your NFT Authentication Journey Starts Now! 

Welcome to AuthiChain, [username]!

Thank you for joining the leading NFT authentication and verification marketplace.

Get Started in 3 Easy Steps:

1. Choose Your Subscription - Select the plan that fits your needs
2. Mint Your First NFT - Create authenticated digital assets
3. Explore the Marketplace - Discover and trade verified NFTs

[Start Your Journey Button]

What You Can Do with AuthiChain:

- ☒ Mint authenticated NFTs with IPFS storage
- ☒ Participate in auctions and bidding
- ☒ Create and manage collections
- ☒ Access advanced analytics and insights
- ☒ Connect with a global community

2. Subscription Confirmation Email (subscription-confirmation-email.tsx)

Purpose: Sent when a user successfully subscribes to a paid plan

Variables:

- username (string): User's display name
- tier ('BASIC' | 'PRO' | 'ENTERPRISE'): Selected subscription tier
- amount (number): Monthly subscription amount
- periodEnd (string): Next billing date

Content Includes:

- Subscription confirmation
- Plan features list
- Billing information
- Quick action buttons
- Usage tips
- Subscription management link

Sample Content:

Subject: Your [tier] Subscription is Active!

Hi [username], your [tier] subscription is now active. Welcome to the next level of NFT authentication!

[Plan Name]
\$[amount]/month

Your Benefits:

☒ [Feature list based on tier]

Next billing date: [periodEnd]

Status: Active

[Go to Dashboard] [Mint Your First NFT]

Make the Most of Your Subscription:

-  Create Collections: Organize your NFTs into themed collections
-  Track Analytics: Monitor your NFT performance and engagement
-  List on Marketplace: Sell your authenticated NFTs to collectors
-  Join Auctions: Participate **in** exclusive auction events

3. First NFT Success Email (`first-nft-success-email.tsx`)

Purpose: Sent when a user successfully mints their first NFT

Variables:

- `username` (string): User's display name
- `nftTitle` (string): Title of the minted NFT
- `nftImage` (string, optional): URL to NFT image
- `nftUrl` (string): Link to view the NFT on platform
- `ipfsUrl` (string): IPFS gateway link

Content Includes:

- Celebration message
- NFT preview
- IPFS and blockchain verification badges
- Next steps suggestions
- Pro tips for success
- Resource links

Sample Content:

Subject: Congratulations on Your First NFT! 🎉

You Did It, [username]!

Congratulations on minting your first authenticated NFT on AuthiChain!

[NFT Image Preview]

[NFT Title]

- ✓ Permanently stored on IPFS
- ✓ Verified and authenticated
- ✓ Successfully minted

[View Your NFT] [View on IPFS]

What's Next? 🚀

- 1️⃣ Share Your Creation
Show the world your authenticated NFT on social media
- 2️⃣ Build a Collection
Create a themed collection to organize your NFTs
- 3️⃣ List on Marketplace
Make your NFT available **for** sale or auction

Pro Tips **for** Success 💡

- Optimize your metadata: Add detailed descriptions and tags
- Engage with collectors: Respond to comments and offers
- Track analytics: Monitor views, likes, and engagement
- Join the community: Connect with other creators and collectors

Sending Emails

Use the helper functions in `/app/lib/email.ts`:

```
import {
  sendWelcomeEmail,
  sendSubscriptionConfirmationEmail,
  sendFirstNFTSuccessEmail
} from '@lib/email';

// Send welcome email
await sendWelcomeEmail(
  'user@example.com',
  'John Doe',
  '0x1234...5678'
);

// Send subscription confirmation
await sendSubscriptionConfirmationEmail(
  'user@example.com',
  'John Doe',
  'PRO',
  99,
  'January 20, 2026'
);

// Send first NFT success
await sendFirstNFTSuccessEmail(
  'user@example.com',
  'John Doe',
  'My First NFT',
  'https://upload.wikimedia.org/wikipedia/commons/thumb/0/09/NFT_diagram.svg/1200px-NFT_diagram.svg.png',
  'https://placeholder.co/1200x600/e2e8f0/1e293b?text=Image_of_the_NFT_titled_My_First_NFT_associated_',
  'https://upload.wikimedia.org/wikipedia/commons/thumb/b/be/VeKings.png/250px-VeKings.png'
);
```

Email Service Configuration

Current Setup: Console logging (development mode)

Production Integration:

To integrate with a production email service (SendGrid, AWS SES, etc.), update `/app/lib/email.ts`:

```
// Example: SendGrid integration
import sgMail from '@sendgrid/mail';

class SendGridEmailService implements EmailService {
  constructor() {
    sgMail.setApiKey(process.env.SENDGRID_API_KEY!);
  }

  async sendEmail(to: string, subject: string, html: string): Promise<void> {
    await sgMail.send({
      to,
      from: 'noreply@authichain.app',
      subject,
      html,
    });
  }
}

function getEmailService(): EmailService {
  if (process.env.NODE_ENV === 'production') {
    return new SendGridEmailService();
  }
  return new ConsoleEmailService();
}
```

In-App Tutorial

Overview

The interactive tutorial uses React Joyride to guide users through key platform features with step-by-step tooltips and instructions.

Component Location

```
/app/components/onboarding/OnboardingTutorial.tsx
```

Tutorial Steps

The tutorial includes 8 steps:

1. **Welcome Message** (center screen)
 - Introduction to AuthiChain
 - Overview of what's coming
2. **Wallet Connection** (header wallet button)
 - How to connect Web3 wallet
 - Importance of wallet connection
3. **Pricing Plans** (pricing link)
 - Overview of subscription tiers
 - How to choose a plan
4. **Minting NFTs** (mint link)
 - Introduction to NFT minting
 - IPFS storage benefits

5. **Marketplace** (marketplace link)
 - Browsing and discovering NFTs
 - Trading functionality
6. **Collections** (collections link)
 - Creating and managing collections
 - Organizing NFTs
7. **Dashboard** (dashboard link)
 - Profile management
 - Analytics and insights
8. **Completion** (center screen)
 - Congratulations message
 - Link to Getting Started guide

Implementation

Add the tutorial component to your layout or dashboard:

```
import { OnboardingTutorial } from '@components/onboarding/OnboardingTutorial';

export default function Dashboard() {
  const { user, isFirstTime } = useUser(); // Your user context

  return (
    <>
      <OnboardingTutorial
        userId={user.id}
        isFirstTime={isFirstTime}
      />
      { /* Rest of your page */ }
    </>
  );
}
```

Data Attributes

Ensure your navigation elements have the correct `data-tour` attributes:

```
<button data-tour="wallet-connect">Connect Wallet</button>
<Link href="/pricing" data-tour="pricing">Pricing</Link>
<Link href="/mint" data-tour="mint">Mint NFT</Link>
<Link href="/marketplace" data-tour="marketplace">Marketplace</Link>
<Link href="/collections" data-tour="collections">Collections</Link>
<Link href="/dashboard" data-tour="dashboard">Dashboard</Link>
```

Customization

Modify the steps array in `OnboardingTutorial.tsx` :

```
const steps: Step[] = [
  {
    target: 'body',
    content: (
      <div>
        <h2>Your Custom Title</h2>
        <p>Your custom content</p>
      </div>
    ),
    placement: 'center',
    disableBeacon: true,
  },
  // Add more steps...
];
```

Styling

The tutorial uses custom styling that matches AuthiChain's branding:

```
styles={{
  options: {
    primaryColor: '#8b5cf6', // Purple brand color
    zIndex: 10000,
  },
  tooltip: {
    borderRadius: 12,
    padding: 20,
  },
  // ... more styles
}}
```

Progress Tracking

Database Schema

The User model includes onboarding tracking fields:

```
model User {
  // ... existing fields

  // Onboarding tracking
  onboardingCompleted Boolean @default(false)
  tutorialCompleted Boolean @default(false)
  walletConnected Boolean @default(false)
  firstNFTMinted Boolean @default(false)
  firstPurchaseMade Boolean @default(false)
  onboardingStep Int @default(0)
}
```

API Endpoints

Get Onboarding Status

```
GET /api/user/onboarding/status?userId={userId}
```

Response:

```
{
  "walletConnected": true,
  "subscriptionActive": true,
  "firstNFTMinted": false,
  "tutorialCompleted": true,
  "profileCompleted": true,
  "onboardingCompleted": false,
  "onboardingStep": 2
}
```

Update Onboarding Status

```
POST /api/user/onboarding/status
Content-Type: application/json
```

```
{
  "userId": "user_id",
  "field": "walletConnected",
  "value": true
}
```

Valid fields:

- walletConnected
- firstNFTMinted
- tutorialCompleted
- onboardingCompleted
- onboardingStep

Complete Tutorial

```
POST /api/user/onboarding/complete-tutorial
Content-Type: application/json
```

```
{
  "userId": "user_id"
}
```

Onboarding Checklist Component

Location: `/app/components/onboarding/OnboardingChecklist.tsx`

Usage**Full Version:**

```
import { OnboardingChecklist } from '@components/onboarding/OnboardingChecklist';

<OnboardingChecklist userId={user.id} />
```

Compact Version (for sidebars):

```
<OnboardingChecklist userId={user.id} compact={true} />
```

With Initial Status (avoid extra API call):

```
<OnboardingChecklist
  userId={user.id}
  initialStatus={{
    walletConnected: true,
    subscriptionActive: false,
    firstNFTMinted: false,
    tutorialCompleted: true,
    profileCompleted: true,
  }}
/>
```

Features

- **Progress Bar:** Visual representation of completion percentage
- **Task List:** Five key onboarding tasks with status indicators
- **Action Buttons:** Direct links to complete pending tasks
- **Celebration:** Special message when all tasks are completed
- **Auto-Refresh:** Polls for status updates
- **Responsive:** Works on all screen sizes

Tasks Tracked

1. **Connect Your Wallet** → `/dashboard`
2. **Choose a Subscription** → `/pricing`
3. **Complete Your Profile** → `/dashboard/settings`
4. **Take the Tour** (Tutorial)
5. **Mint Your First NFT** → `/mint`

Getting Started Page

Location

`/app/app/getting-started/page.tsx`

Overview

A comprehensive, self-serve guide page with:

- Quick action links
- Tabbed content sections
- Step-by-step tutorials
- Video placeholders
- FAQ accordions
- Best practices
- Support resources

Content Sections

1. Wallet Setup

- MetaMask installation guide
- Wallet creation steps

- Connection process
- Supported wallets
- Security tips
- FAQ

2. Subscriptions

- Plan comparison cards
- Feature breakdowns
- Pricing details
- Payment methods
- Link to full pricing page

3. Minting NFTs

- Complete minting guide
- File format requirements
- IPFS explanation
- Metadata best practices
- Quality guidelines
- Success tips

4. Marketplace

- Buying process
- Selling process
- Auction system
- Search and filtering
- Safety tips

5. Advanced Features

- Collections management
- Analytics overview
- API integration
- White-label solution
- Enterprise features

Video Tutorial Placeholders

The page includes placeholders for video tutorials:

```
const videoPlaceholders = [
  { title: 'Welcome to AuthiChain', duration: '2:30', thumbnail: '🎬' },
  { title: 'Connecting Your Wallet', duration: '3:15', thumbnail: '👛' },
  { title: 'Minting Your First NFT', duration: '5:45', thumbnail: '🎨' },
  { title: 'Using the Marketplace', duration: '4:20', thumbnail: '🛒' },
];
```

To add real videos:

1. Host videos on YouTube, Vimeo, or self-host
2. Replace the placeholder div with an iframe or video player
3. Update the thumbnail with actual video thumbnail images

Example:

```
<iframe
  width="100%"
  height="200"
  src="https://www.youtube.com/embed/YOUR_VIDEO_ID"
  frameborder="0"
  allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope;
  picture-in-picture"
  allowFullScreen
></iframe>
```

Customization

The page is fully customizable via the React component. Key areas to customize:

1. **Quick Links:** Update the `quickLinks` array
2. **Tab Content:** Modify `TabsContent` sections
3. **Step-by-Step Guides:** Edit the numbered instruction blocks
4. **Support Channels:** Update contact information and links

Implementation Guide

Step 1: Database Migration

Run the Prisma migration to add onboarding fields:

```
cd /home/ubuntu/authichain_premium/app
npx prisma migrate dev --name add_onboarding_fields
npx prisma generate
```

Step 2: Integration Points

A. User Registration

After creating a new user, send the welcome email:

```
// In your registration handler
const user = await prisma.user.create({ /* ... */ });

// Send welcome email
await sendWelcomeEmail(
  user.email,
  user.firstName,
  user.walletAddress
);
```

B. Wallet Connection

When a user connects their wallet, update the status:

```
// In your wallet connection handler
await fetch('/api/user/onboarding/status', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    userId: user.id,
    field: 'walletConnected',
    value: true,
  }),
});
```

C. Subscription Purchase

After successful subscription, send confirmation email:

```
// In your Stripe webhook handler
await sendSubscriptionConfirmationEmail(
  user.email,
  user.firstName,
  subscription.tier,
  subscription.amount / 100, // Convert cents to dollars
  new Date(subscription.currentPeriodEnd).toLocaleDateString()
);
```

D. First NFT Mint

When a user mints their first NFT, send success email and update status:

```
// In your NFT minting handler
const isFirstNFT = await prisma.nftUpload.count({
  where: { userId: user.id }
}) === 1;

if (isFirstNFT) {
  await sendFirstNFTSuccessEmail(
    user.email,
    user.firstName,
    nft.title,
    nft.imageUrl,
    `${process.env.NEXT_PUBLIC_APP_URL}/nft/${nft.id}`,
    `https://filecoin.io/uploads/screen-shot-2021-04-29-at-1-51-42-pm.png`
  );

  await fetch('/api/user/onboarding/status', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      userId: user.id,
      field: 'firstNFTMinted',
      value: true,
    }),
  });
}
```

E. Dashboard Integration

Add the checklist and tutorial to your dashboard:

```
// app/dashboard/page.tsx
import { OnboardingTutorial } from '@components/onboarding/OnboardingTutorial';
import { OnboardingChecklist } from '@components/onboarding/OnboardingChecklist';

export default function DashboardPage() {
  const { user } = useUser();

  return (
    <div>
      <OnboardingTutorial
        userId={user.id}
        isFirstTime={!user.tutorialCompleted}
      />

      {!user.onboardingCompleted && (
        <div className="mb-8">
          <OnboardingChecklist userId={user.id} />
        </div>
      )}

      { /* Rest of dashboard */ }
    </div>
  );
}
```

Step 3: Add Navigation Data Attributes

Update your main navigation to include data-tour attributes:

```
// components/Header.tsx or similar
<nav>
  <Link href="/pricing" data-tour="pricing">Pricing</Link>
  <Link href="/mint" data-tour="mint">Mint</Link>
  <Link href="/marketplace" data-tour="marketplace">Marketplace</Link>
  <Link href="/collections" data-tour="collections">Collections</Link>
  <Link href="/dashboard" data-tour="dashboard">Dashboard</Link>
  <button data-tour="wallet-connect" onClick={connectWallet}>
    Connect Wallet
  </button>
</nav>
```

Step 4: Test the System

1. Create a new test user account
 2. Verify welcome email is logged/sent
 3. Check that the tutorial launches on first login
 4. Test the onboarding checklist displays correctly
 5. Connect a wallet and verify status updates
 6. Subscribe to a plan and verify confirmation email
 7. Mint an NFT and verify success email
 8. Confirm all checklist items can be completed
-

Customization

Branding

All components use AuthiChain's purple/pink color scheme. To customize:

Tutorial Colors:

```
// In OnboardingTutorial.tsx
styles={{
  options: {
    primaryColor: '#YOUR_COLOR', // Replace with your brand color
  },
  // ...
}}
```

Email Colors:

```
// In email template files
const button = {
  backgroundColor: '#YOUR_COLOR', // Replace with your brand color
  // ...
};
```

Checklist Colors:

Use Tailwind classes in OnboardingChecklist.tsx:

- `bg-purple-600` → `bg-your-color`
- `text-purple-600` → `text-your-color`
- `border-purple-600` → `border-your-color`

Content

Email Content

Edit the JSX content in each email template file:

- `/app/emails/welcome-email.tsx`
- `/app/emails/subscription-confirmation-email.tsx`
- `/app/emails/first-nft-success-email.tsx`

Tutorial Steps

Modify the `steps` array in `OnboardingTutorial.tsx`

Getting Started Page

Edit the content sections in `/app/app/getting-started/page.tsx`

Checklist Tasks

Update the `tasks` array in `OnboardingChecklist.tsx`

Adding New Tutorial Steps

```
// In OnboardingTutorial.tsx, add to steps array:
{
  target: '[data-tour="your-feature"]',
  content: (
    <div>
      <h3 className="text-xl font-bold mb-2">Your Feature</h3>
      <p>Description of your feature</p>
    </div>
  ),
  placement: 'bottom', // or 'top', 'left', 'right', 'center'
},
```

Then add the data attribute to your component:

```
<YourComponent data-tour="your-feature" />
```

Adding New Checklist Tasks

```
// In OnboardingChecklist.tsx, add to tasks array:
{
  id: 'your-task',
  title: 'Your Task Title',
  description: 'What the user should do',
  completed: status.yourTaskField,
  action: '/path-to-action',
  icon: '🔥',
}
```

Update the status interface and API to track the new field.

Analytics & Monitoring

Key Metrics to Track

1. **Onboarding Completion Rate**
 - % of users who complete all checklist items
 - Time to complete onboarding
2. **Tutorial Engagement**
 - % of users who start the tutorial
 - % of users who complete the tutorial
 - % of users who skip the tutorial
 - Average completion time
3. **Email Performance**
 - Open rates for each email type
 - Click-through rates
 - Conversion rates (email → action)

4. Feature Adoption

- % of users who connect wallet
- % of users who subscribe
- % of users who mint first NFT
- Time from registration to first mint

5. Drop-off Points

- Where users stop in the onboarding process
- Common tutorial exit points

Database Queries

Onboarding Completion Rate

```
SELECT
  COUNT(CASE WHEN "onboardingCompleted" = true THEN 1 END)::float / COUNT(*) * 100 as
  completion_rate
FROM "User"
WHERE "createdAt" >= NOW() - INTERVAL '30 days';
```

Average Time to First NFT

```
SELECT
  AVG(EXTRACT(EPOCH FROM (first_nft."createdAt" - u."createdAt")) / 3600) as avg_hours
FROM "User" u
JOIN LATERAL (
  SELECT "createdAt"
  FROM "NftUpload"
  WHERE "userId" = u.id
  ORDER BY "createdAt"
  LIMIT 1
) first_nft ON true
WHERE u."firstNFTMinted" = true;
```

Tutorial Completion by Cohort

```
SELECT
  DATE_TRUNC('week', "createdAt") as week,
  COUNT(*) as total_users,
  COUNT(CASE WHEN "tutorialCompleted" = true THEN 1 END) as completed_tutorial,
  COUNT(CASE WHEN "tutorialCompleted" = true THEN 1 END)::float / COUNT(*) * 100 as co
  mpletion_rate
FROM "User"
GROUP BY DATE_TRUNC('week', "createdAt")
ORDER BY week DESC
LIMIT 12;
```

Analytics Integration

Add tracking events for key onboarding actions:

```
// Example with Google Analytics
import { gtag } from '@lib/analytics';

// Track tutorial start
gtag('event', 'tutorial_begin', {
  event_category: 'onboarding',
});

// Track tutorial completion
gtag('event', 'tutorial_complete', {
  event_category: 'onboarding',
});

// Track checklist task completion
gtag('event', 'onboarding_task_complete', {
  event_category: 'onboarding',
  event_label: 'wallet_connected',
});

// Track onboarding completion
gtag('event', 'onboarding_complete', {
  event_category: 'onboarding',
  value: daysToComplete,
});
```

Troubleshooting

Common Issues

1. Tutorial Not Starting

Issue: Tutorial doesn't appear for new users

Solutions:

- Check that `isFirstTime` prop is correctly passed
- Verify `tutorialCompleted` field is false in database
- Ensure data-tour attributes are present on navigation elements
- Check browser console for JavaScript errors

2. Checklist Not Updating

Issue: Checklist doesn't reflect completed tasks

Solutions:

- Verify API endpoints are returning correct data
- Check that database fields are being updated
- Ensure `userId` is correctly passed to component
- Check browser network tab for failed API calls

3. Emails Not Sending

Issue: Users not receiving emails

Solutions:

- Verify email service is configured (currently console logging)
- Check that email functions are called at correct points

- Ensure user email address is valid
- Check spam folders (for production)
- Verify email templates render correctly

4. Progress Not Persisting

Issue: User progress resets

Solutions:

- Verify database migrations ran successfully
- Check that API is updating correct user record
- Ensure user session is maintained
- Check for database connection issues

Debug Mode

Enable debug logging by adding to your components:

```
// In OnboardingTutorial.tsx
useEffect(() => {
  console.log('Tutorial props:', { userId, isFirstTime, run });
}, [userId, isFirstTime, run]);

// In OnboardingChecklist.tsx
useEffect(() => {
  console.log('Checklist status:', status);
}, [status]);
```

Future Enhancements

Planned Features

1. Follow-up Email Sequences

- Reminder emails for incomplete onboarding
- Re-engagement campaigns
- Feature announcement emails

2. Personalized Onboarding Paths

- Different flows for creators vs collectors
- Role-based tutorial steps
- Industry-specific guides

3. Gamification

- XP points for completed tasks
- Badges and achievements
- Leaderboard for early adopters

4. Video Tutorials

- Record and embed actual tutorial videos
- Interactive video elements
- Closed captions and transcripts

5. In-App Messaging

- Contextual help tooltips
- Announcement banners
- Feature discovery prompts

6. A/B Testing

- Test different onboarding flows
- Optimize email content
- Experiment with tutorial steps

7. Advanced Analytics

- Funnel visualization
- Cohort analysis
- Predictive churn modeling

Contributing

To add new onboarding features:

1. Create a new branch for your feature
2. Implement the feature following existing patterns
3. Update this documentation
4. Add tests if applicable
5. Submit a pull request

Support

For questions or issues with the onboarding system:

- **Technical Support:** support@authichain.app
- **Documentation:** <https://authichain.app/docs>
- **Developer Portal:** <https://developers.authichain.app>

Changelog

Version 1.0.0 (October 2025)

- Initial onboarding system implementation
 - Three email templates
 - Interactive tutorial with React Joyride
 - Progress tracking system
 - Onboarding checklist component
 - Comprehensive getting started page
 - API endpoints for status tracking
 - Database schema with onboarding fields
-

Last Updated: October 20, 2025

Maintained by: AuthiChain Development Team

Version: 1.0.0